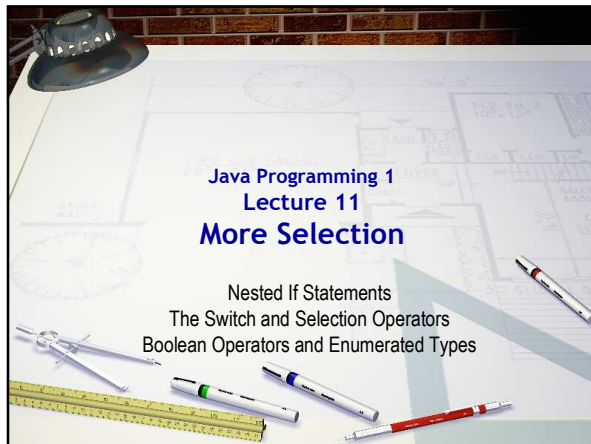
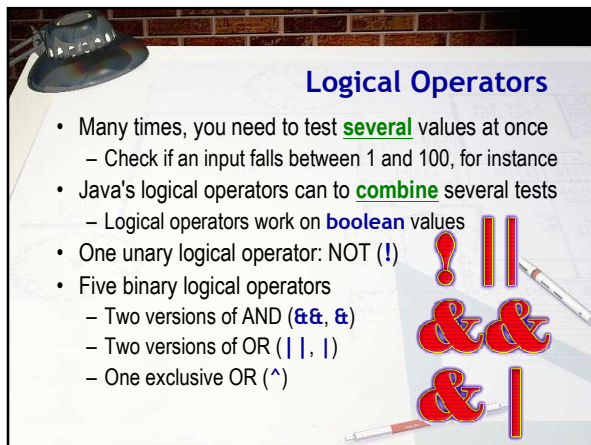
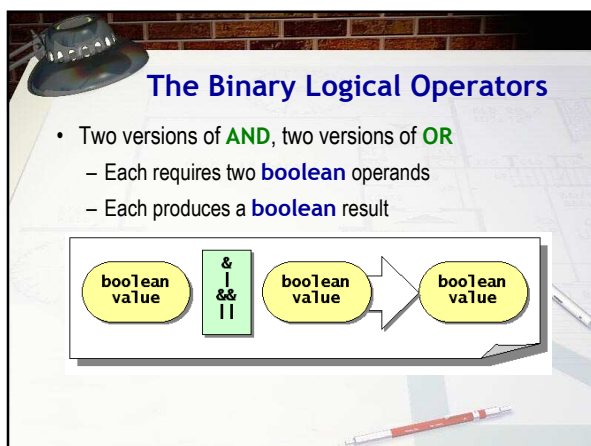
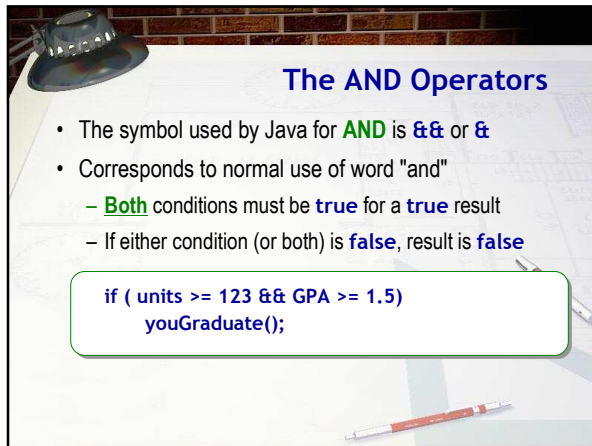






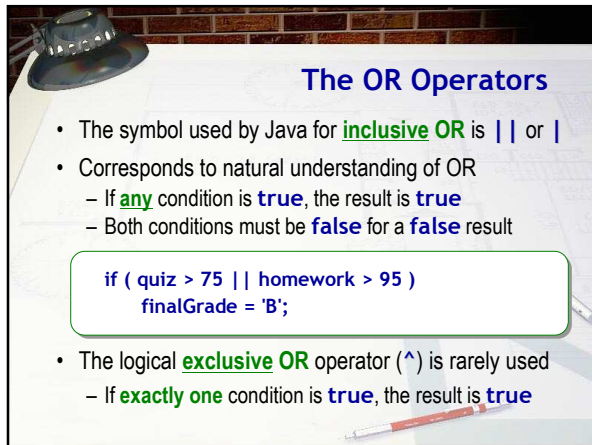




The AND Operators

- The symbol used by Java for **AND** is `&&` or `&`
- Corresponds to normal use of word "and"
 - **Both** conditions must be **true** for a **true** result
 - If either condition (or both) is **false**, result is **false**

```
if ( units >= 123 && GPA >= 1.5 )
    youGraduate();
```

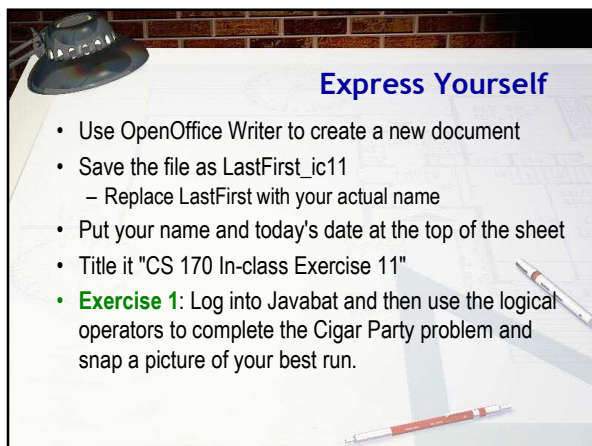


The OR Operators

- The symbol used by Java for **inclusive OR** is `||` or `|`
- Corresponds to natural understanding of OR
 - If **any** condition is **true**, the result is **true**
 - Both conditions must be **false** for a **false** result

```
if ( quiz > 75 || homework > 95 )
    finalGrade = 'B';
```

- The logical **exclusive OR** operator (^) is rarely used
 - If **exactly one** condition is **true**, the result is **true**



Express Yourself

- Use OpenOffice Writer to create a new document
- Save the file as LastFirst_ic11
 - Replace LastFirst with your actual name
- Put your name and today's date at the top of the sheet
- Title it "CS 170 In-class Exercise 11"
- **Exercise 1:** Log into Javabat and then use the logical operators to complete the Cigar Party problem and snap a picture of your best run.

The Impossible Condition

- When no combination of operands could ever be true
 - "Give a 25% discount to children who are under 13 **and** to patrons who are 65 and older"
- A "literal" reading results in code like this:

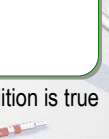
```
if ( age < 13 && age > 65 )
    discount = .25;
else
    discount = 0;
```
- No possible value for **age** where this could be true



The Unavoidable Condition

- When no conceivable condition could **ever** be false
 - "If the patron's age is greater than 12 **or** less than 66, charge 8.75 for the ticket. Everyone else pays 5.75."
- A "literal" translation results in code like this:

```
if (age > 12 || age < 66)
    ticketPrice = 8.75;
else
    ticketPrice = 5.75;
```
- For **every possible value** of **age**, the condition is true

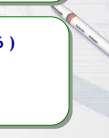


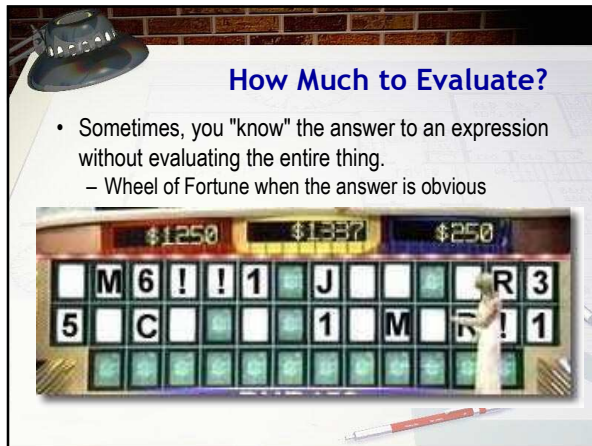
Solutions

- Both conditions caused by **confusing AND and OR**
- First example **should** look like this:

```
if ( age < 13 || age > 65 )
    discount = .25;
else
    discount = 0;
```
- Second example should look like this:

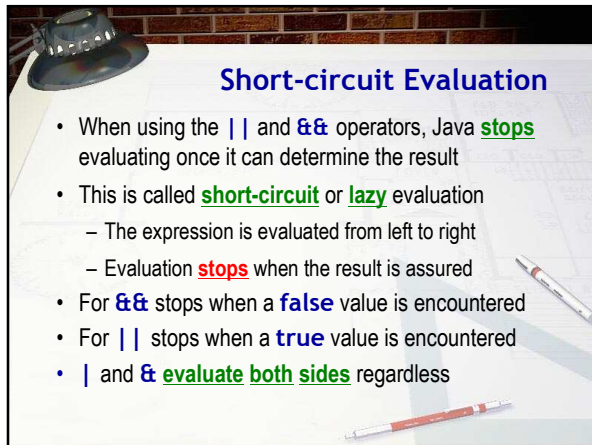
```
if ( age > 12 && age < 66 )
    ticketPrice = 8.75;
else
    ticketPrice = 5.75;
```





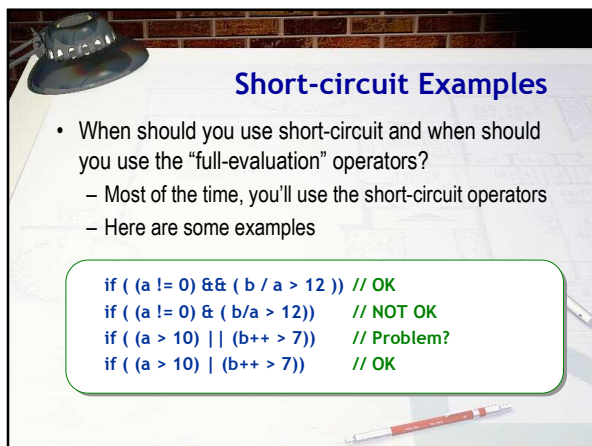
How Much to Evaluate?

- Sometimes, you "know" the answer to an expression without evaluating the entire thing.
 - Wheel of Fortune when the answer is obvious



Short-circuit Evaluation

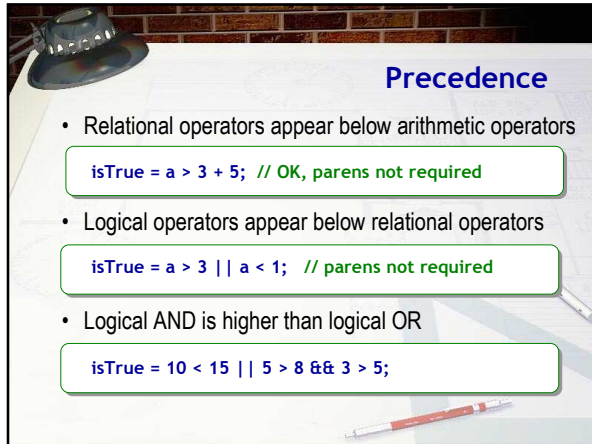
- When using the `||` and `&&` operators, Java **stops** evaluating once it can determine the result
- This is called **short-circuit** or **lazy** evaluation
 - The expression is evaluated from left to right
 - Evaluation **stops** when the result is assured
- For `&&` stops when a **false** value is encountered
- For `||` stops when a **true** value is encountered
- `|` and `&` **evaluate both sides** regardless



Short-circuit Examples

- When should you use short-circuit and when should you use the "full-evaluation" operators?
 - Most of the time, you'll use the short-circuit operators
 - Here are some examples

```
if ( (a != 0) && (b / a > 12) ) // OK
if ( (a != 0) & (b/a > 12) )    // NOT OK
if ( (a > 10) || (b++ > 7) )    // Problem?
if ( (a > 10) | (b++ > 7) )      // OK
```



Precedence

- Relational operators appear below arithmetic operators

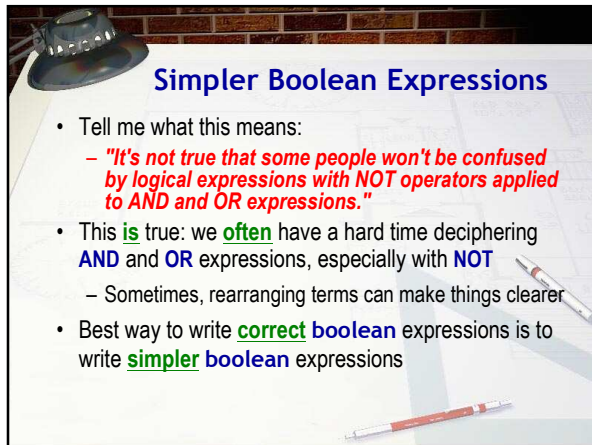
```
isTrue = a > 3 + 5; // OK, parens not required
```

- Logical operators appear below relational operators

```
isTrue = a > 3 || a < 1; // parens not required
```

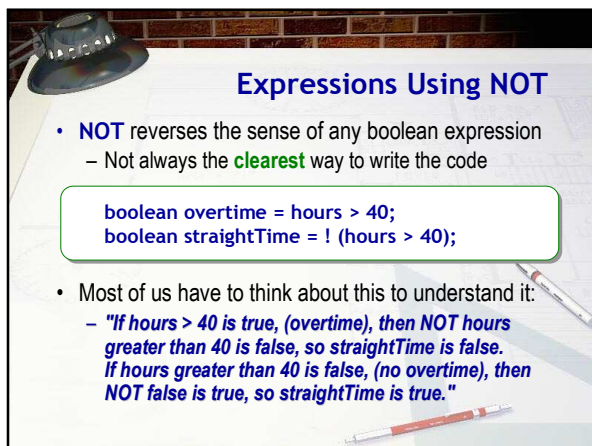
- Logical AND is higher than logical OR

```
isTrue = 10 < 15 || 5 > 8 && 3 > 5;
```



Simpler Boolean Expressions

- Tell me what this means:
 - "It's not true that some people won't be confused by logical expressions with NOT operators applied to AND and OR expressions."*
- This **is** true: we **often** have a hard time deciphering **AND** and **OR** expressions, especially with **NOT**
 - Sometimes, rearranging terms can make things clearer
- Best way to write **correct boolean** expressions is to write **simpler boolean** expressions



Expressions Using NOT

- NOT** reverses the sense of any boolean expression
 - Not always the **clearest** way to write the code

```
boolean overtime = hours > 40;  
boolean straightTime = ! (hours > 40);
```

- Most of us have to think about this to understand it:
 - "If hours > 40 is true, (overtime), then NOT hours greater than 40 is false, so straightTime is false. If hours greater than 40 is false, (no overtime), then NOT false is true, so straightTime is true."*

Alternatives to NOT

- Most of us find this way of writing the example clearer:


```
boolean straightTime = hours <= 40;
```
- Relational operators let you avoid **NOT** like this

NOT Expression	Equivalent
<code>! (a == b)</code>	<code>a != b</code>
<code>! (a > b)</code>	<code>a <= b</code>
<code>! (a < b)</code>	<code>a >= b</code>
<code>! (a <= b)</code>	<code>a > b</code>
<code>! (a >= b)</code>	<code>a < b</code>

NOT, AND, and OR

- NOT** becomes even more complex when combined with the other logical operators, **AND** and **OR**
 - Test for those who can legally drink but who don't yet qualify for Social Security

```
if (age >= 21 && age < 65) ...
```

– Now consider its negation

```
if ( ! (age >= 21 && age < 65) ) ...
```

– Again, most of us have to **think** about it

De Morgan's Law

- Designed to simplify these kinds of expressions
 - `! (A && B)` is equivalent to `!A || !B`
 - `! (A || B)` is equivalent to `!A && !B`
- Convert our previous example like this:


```
if ( ! (age >= 21) || ! (age < 65) ) ...
```

– then, using alternatives to **NOT**, we get

```
if ( age < 21 || age >= 65 ) ...
```

Express Yourself

- Exercise 2:** Simplify these Boolean expressions
 - $\neg (x > 0 \ \&\& \ y > 0)$
 - $\neg (x \neq 0 \ || \ y \neq 0)$
 - $\neg (\text{country.equals("US")} \ \&\& \ \text{!state.equals("HI")} \ \&\& \ \text{!state.equals("AK")})$
 - $\neg (x\%4 \neq 0 \ || \ \text{!(x \% 100 == 0 \ \&\& \ x \% 400 == 0)})$

Multi-Way Branching

- Many problems involve branching statements with **more** than just the two alternatives handled by **if-else**
 - Calculating grades based on a numeric score
 - Income taxes based on filing status and income

Student and TestStudent

- Exercise 3:** Download **ic11.zip** and open using Open Non-BlueJ. Compile and run **TestStudent**. Shoot a screen-shot. We'll explore different ways to solve.
 - Assign letter grades based on numeric scores
- Note Student is finished except for this part

Inside TestStudent

- **TestStudent** creates a **Student** and adds some tests
- Then **getPercentage()** is used to return current score
- Finally, **getLetterGrade()** prints letter grade

```

26:
27: // 3. Print Fred's letter grade
28: System.out.println("Fred's letter grade is : " +
29:     fred.getLetterGrade());
30: }
31: }

17: fred.addScore(10, 100);
18: fred.addScore(15, 15);

```

Inside Student

- Fields **name**, **totalPoints**, **studentPoints**
- Mutator **addScore()** modifies student and total points
- Accessor **getPercentage()** calculates numeric grade
- Accessor **getLetterGrade()** is incomplete

```

46: /**
47:  * Calculate the current letter grade
48:  * @return current letter grade using multi-way selection
49:  */
50: public String getLetterGrade()
51: {
52:     return "";
53: }
54: }

44: }
45:

```

Independent if Statements

```

46: /**
47:  * Calculate the current letter grade
48:  * @return current letter grade using multi-way selection
49:  */
50: public String getLetterGrade()
51: {
52:     double percent = getPercentage();
53:     String result = "";
54:
55:     if (percent >= .90 && percent <= 1.0)
56:         result = "A";
57:     if (percent >= .80 && percent < .90)
58:         result = "B";
59:     if (percent >= .70 && percent < .80)
60:         result = "C";
61:     if (percent >= .60 && percent < .70)
62:         result = "D";
63:     if (percent >= 0.0 && percent < .60)
64:         result = "F";
65:     if (percent < 0.0 || percent > 1.0)
66:         result = "Out of range";
67:     return result;
68: }
69: }
70: }

```

Multiple Selection

- **Independent** ifs, but **interdependent** conditions
 - **Inefficient**: all six conditions are tested each time
- Better: “**Multiple Selection Algorithmic Pattern**”
 - “Pattern” is a **recipe** for solving a particular category of problem
 - AKA as “ladder-style” if-else statements
- **Pitfall**: order is significant

```
if ( condition-1 is true )
    execute action-1
else if ( condition-2 is true )
    execute action-2
// ...
else if ( condition n-1 is true )
    execute action n-1
else
    execute action-n
```

Express Yourself

- **Exercise 4**: Multiple-selection pattern
 - Modify **getLetterGrade()** to use this pattern
 - Shoot a screen shot of your method
- Questions to ask
 - What range do you start with?
 - When and how do you handle “out of range” values like those less than 0.0 or over 100.00?

Nested if Statements

- Can also combine Boolean expressions with **nesting**
 - One **if** statement embedded inside the body of another

```
if ( x > 0 )
{
    if ( y <= 0 )
        quadrant = SIRIUS;
    else
        quadrant = OMEGA;
}
```

A nested if statement

- The nested **if** can appear in the **else** part as well

Using Nested if Statements

- Use nested **ifs** when you have **levels** of decisions
 - Calculating income tax, for instance
 - First decide on filing status, then decide on income

Filing Status : Single		Filing Status: Married Filing Jointly	
Taxable Income	Tax	Taxable Income	Tax
\$100,000 ... \$146,750	20% less \$5,373	\$100,000 ... \$117,250	25% less \$6,525
\$146,751 ... \$319,100	33% less \$12,710.50	\$117,251 ... \$178,650	20% less \$10,042.50
Over \$319,100	35% less \$19,092.50	\$178,651 ... \$319,100	33% less \$18,975
		Over \$319,100	35% less \$26,367
Filing Status : Married Filing Separately		Filing Status: Head of Household	
Taxable Income	Tax	Taxable Income	Tax
\$100,000 ... \$159,550	33% less \$9,487.50	\$100,000 ... \$100,500	25% less \$4,400
Over \$159,550	35% less \$12,678.50	\$100,501 ... \$162,700	20% less \$7,415
		\$162,701 ... \$319,100	33% less \$18,560
		Over \$319,100	35% less \$21,932

Filing Status

- Use sequential **ifs** to determine filing status

```

if (status == SINGLE)
{
    // calculate the tax for a single taxpayer
}
else if (status == MARRIED_JOINT_FILE)
{
    // calculate for a joint-filing married taxpayer
}
else if (status == MARRIED_SINGLE_FILE)
{
    // calculate for a single-filing married taxpayer
}
else if (status == HEAD_OF_HOUSEHOLD)
{
    // calculate for a head-of-household taxpayer
}

```

Calculating Tax

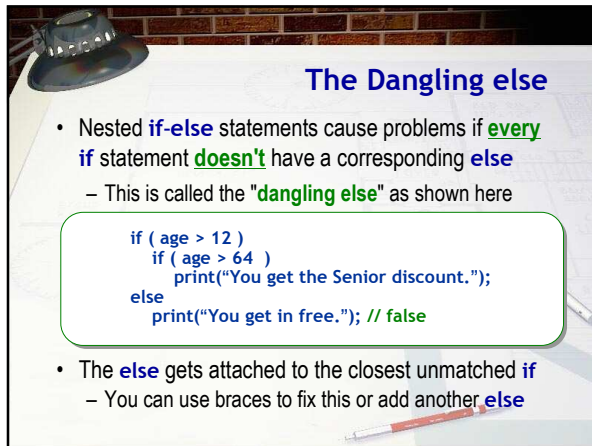
- Use **nested** sequential **ifs** to calculate the tax

```

if (status == SINGLE)
{
    // calculate the tax for a single taxpayer
    if (taxableIncome <= SINGLE_BRACKET1)
        tax = taxableIncome * SINGLE_RATE1 - SINGLE_EX1;
    else if (taxableIncome <= SINGLE_BRACKET2)
        tax = taxableIncome * SINGLE_RATE2 - SINGLE_EX2;
    else
        tax = taxableIncome * SINGLE_RATE3 - SINGLE_EX3;
}
else if (status ==
{
    // calculate t

```

Exercise 5: Modify getLetterGrade() to use nested ifs. Which parts are leveled and which sequential? Shoot a picture of your code.

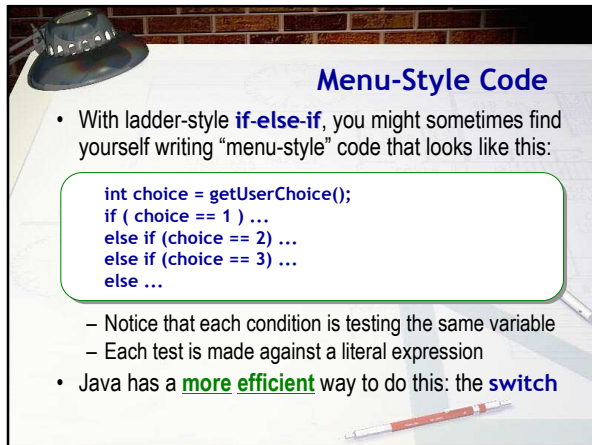


The Dangling else

- Nested **if-else** statements cause problems if **every if** statement **doesn't** have a corresponding **else**
 - This is called the "**dangling else**" as shown here

```
if ( age > 12 )
  if ( age > 64 )
    print("You get the Senior discount.");
  else
    print("You get in free."); // false
```

- The **else** gets attached to the closest unmatched **if**
 - You can use braces to fix this or add another **else**

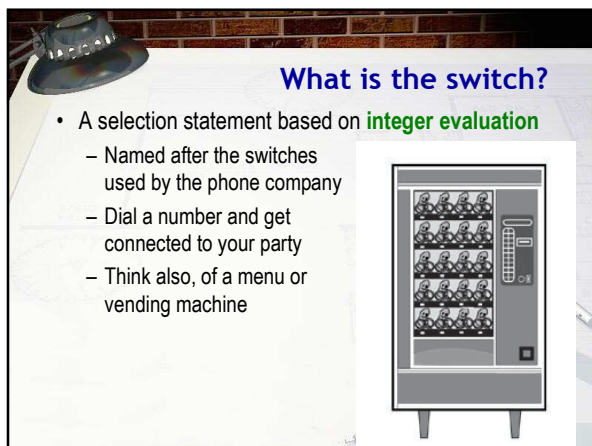


Menu-Style Code

- With ladder-style **if-else-if**, you might sometimes find yourself writing "menu-style" code that looks like this:

```
int choice = getUserChoice();
if ( choice == 1 ) ...
else if ( choice == 2 ) ...
else if ( choice == 3 ) ...
else ...
```

- Notice that each condition is testing the same variable
- Each test is made against a literal expression
- Java has a **more efficient** way to do this: the **switch**



What is the switch?

- A selection statement based on **integer evaluation**
 - Named after the switches used by the phone company
 - Dial a number and get connected to your party
 - Think also, of a menu or vending machine



Parts of the switch

- The parts of a **switch** statement are
 - The **test condition** or **switch selector**
 - A body surrounded by braces (required)
 - A set of labeled code fragments called **case blocks**
 - A "flow-of-control" statement called **break**

The switch Selector

- The **switch selector** can be most integer expressions
 - Includes **int**, **short**, **byte**, and **char**
 - Can't** use **boolean**, **long**, **float**, **double**, or **String**
 - Placed in parentheses following keyword **switch**

Using the switch Statement

```
switch (ch)
{
    case '1':
        aLbl.setText("Here's TOFU");
        break;
    case '2':
        aLbl.setText("Here's TWINKIE");
        break;
    default:
        aLbl.setText("Unknown Choice");
}
```

The Case Block

- Each **case block** begins with the keyword **case**
 - Followed by an **integer constant** and a **colon**
 - Called the **case label**
 - Literals or **static final** constants; no variables, ranges

Using the switch Statement

```
switch (ch)
{
    case '1':
        aLbl.setText("Here's TOFU");
        break;
    case '2':
        aLbl.setText("Here's TWINKIE");
        break;
    default:
        aLbl.setText("Unknown Choice");
}
```

How the switch Works

- When a case label matches the switch selector
 - Java "jumps" to the code immediately following label
- Continues executing **all** the remaining code
 - This is called "**fall through**"
 - Usually put a **break** statement at end of every **case**

Using the switch Statement

```
switch (ch)
{
    case '1':
        albl.setText("Here's TOFU");
        break;
    case '2':
        albl.setText("Here's TWINKIE");
        break;
    default:
        albl.setText("Unknown Choice");
}
```

The default Case

- If the switch selector doesn't match any case?
 - Jump to code past switch or provide a default case
 - Use keyword **default** followed by a colon

Using the switch Statement

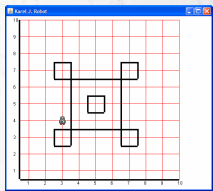
```
switch (ch)
{
    case '1':
        albl.setText("Here's TOFU");
        break;
    case '2':
        albl.setText("Here's TWINKIE");
        break;
    default:
        albl.setText("Unknown Choice");
}
```

Express Yourself

- **Exercise 6:** Using the **switch**. As you learned, you can only use an integer as a **switch** selector, not a **double**. However, you can use casting to turn the grade percentage into an appropriate integer which you can use as a **switch** selector.
 - Modify **getLetterGrade()** to use this pattern
 - Shoot a screen shot of your method

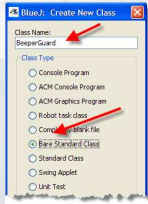
Karel Returns

- When we last saw Karel, he was busy working as a security guard, patrolling the castle at Disneyland
- Exercise 7:** Create a **PatrolTheCastle** object on the workbench and call its **task** method. Snap a picture when Karel is finished working.
- So, how does Karel's boss know that he just didn't come to work and fall asleep?



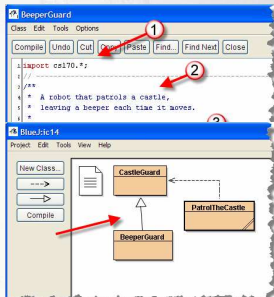
The BeeperGuard

- We created a specialized version of **OCRobot** named **CastleGuard** using **inheritance**. Let's create a new version that drops a beeper each time it moves. That way the boss can check up on him.
- Exercise 8:** add a new Bare Standard Class to your project, and name it **BeeperGuard**. Then, open the file in your editor.



Setting Up BeeperGuard

- Add these changes to **BeeperGuard** and compile:
 - import cs170.*;** on line 1
 - extends CastleGuard** on the header line
 - Add a comment and your name at the top of the class
- Note that **BeeperGuard** is now a specialized kind of a **CastleGuard**



Setting Up the Task

- Add these changes to **PatrolTheCastle**
 - Change **CastleGuard** variable to **BeeperGuard**
 - Initialize karel using the **BeeperGuard()** constructor

```

1  // version Spring 2008
2  //
3  public class PatrolTheCastle {
4  {
5      // Instance/static variables
6      private BeeperGuard karel = null;
7  }
8  }
9  }
10
11 // Instance/static variables
12 private BeeperGuard karel = null;
13
14 // Instance/static variables
15 private BeeperGuard karel = null;
16
17 // Instance/static variables
18 private BeeperGuard karel = null;
19
20 // Instance/static variables
21 private BeeperGuard karel = null;
22
23 // Instance/static variables
24 private BeeperGuard karel = null;
25
26 // Instance/static variables
27 private BeeperGuard karel = null;
28
29 // Instance/static variables
30 private BeeperGuard karel = null;
31
32 // Instance/static variables
33 private BeeperGuard karel = null;
34
35 // Instance/static variables
36 private BeeperGuard karel = null;
37
38 // Instance/static variables
39 private BeeperGuard karel = null;
40
41 // Instance/static variables
42 private BeeperGuard karel = null;
43
44 // Instance/static variables
45 private BeeperGuard karel = null;
46
47 // Instance/static variables
48 private BeeperGuard karel = null;
49
50 // Instance/static variables
51 private BeeperGuard karel = null;
52
53 // Instance/static variables
54 private BeeperGuard karel = null;
55
56 // Instance/static variables
57 private BeeperGuard karel = null;
58
59 // Instance/static variables
60 private BeeperGuard karel = null;
61
62 // Instance/static variables
63 private BeeperGuard karel = null;
64
65 // Instance/static variables
66 private BeeperGuard karel = null;
67
68 // Instance/static variables
69 private BeeperGuard karel = null;
70
71 // Instance/static variables
72 private BeeperGuard karel = null;
73
74 // Instance/static variables
75 private BeeperGuard karel = null;
76
77 // Instance/static variables
78 private BeeperGuard karel = null;
79
80 // Instance/static variables
81 private BeeperGuard karel = null;
82
83 // Instance/static variables
84 private BeeperGuard karel = null;
85
86 // Instance/static variables
87 private BeeperGuard karel = null;
88
89 // Instance/static variables
90 private BeeperGuard karel = null;
91
92 // Instance/static variables
93 private BeeperGuard karel = null;
94
95 // Instance/static variables
96 private BeeperGuard karel = null;
97
98 // Instance/static variables
99 private BeeperGuard karel = null;
100

```

Exercise 9: create a **PatrolTheCastle**, call **task()**

Dropping Beepers

- The **BeeperGuard** already knows how to move, but we want to **extend** its existing behavior. To do that, we have to **override** the **move()** method like this:

```

public void move()
{
    putBeeper();
    super.move();
}

```

Exercise 10: Add the new **move()** method, create a task and patrol the castle. Shoot a screen-shot.

- We call the "parent's" **move()** by using **super**

What's Wrong?

- The **BeeperGuard** has no beepers in its bag
 - Give it some beepers by adding a constructor

```

public BeeperGuard(int st, int ave,
    Direction facing, int beepers )
{
    super(st, ave, facing, beepers);
}

```

- Doesn't compile, since **CastleGuard** has no constructor either. Copy same constructor to **CastleGuard** (change name) and compile.

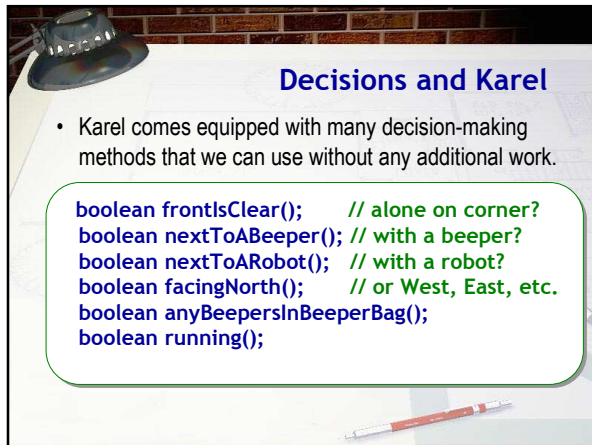


Starting Work

- Now Karel can "teleport" to work. No need for the `goToWork()` method. Comment out the method from `CastleGuard`, comment out the method call from `PatrolTheCastle`, and create karel like this:

```
karel = new BeeperGuard(4, 3, North, infinity);
```

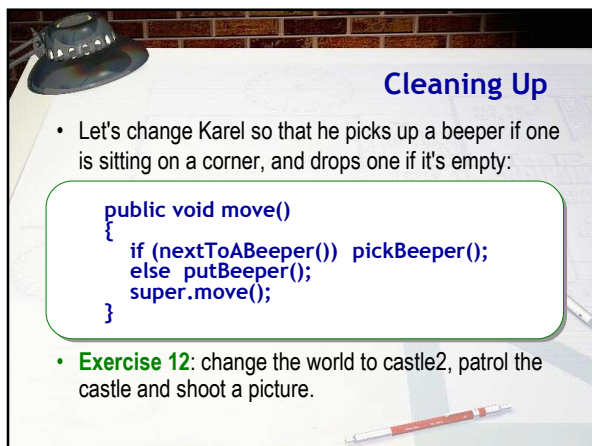
- Exercise 11:** patrol the castle and shoot a picture.
- What happens if Karel goes around twice?



Decisions and Karel

- Karel comes equipped with many decision-making methods that we can use without any additional work.

```
boolean frontIsClear();    // alone on corner?  
boolean nextToABeeper();  // with a beeper?  
boolean nextToARobot();   // with a robot?  
boolean facingNorth();    // or West, East, etc.  
boolean anyBeepersInBeeperBag();  
boolean running();
```

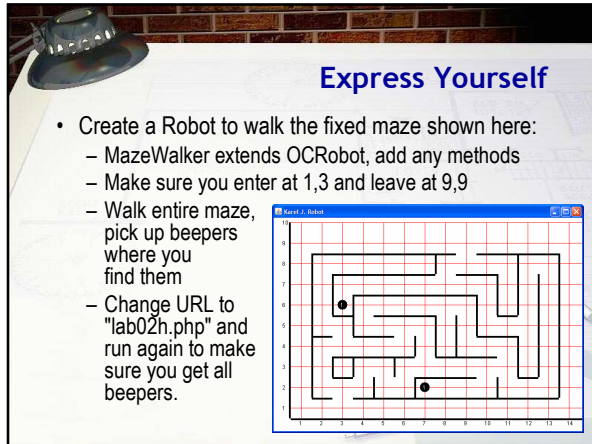


Cleaning Up

- Let's change Karel so that he picks up a beeper if one is sitting on a corner, and drops one if it's empty:

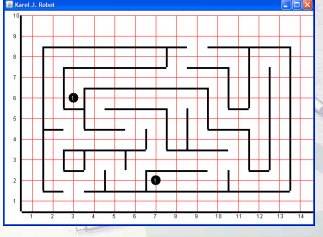
```
public void move()  
{  
    if (nextToABeeper()) pickBeeper();  
    else putBeeper();  
    super.move();  
}
```

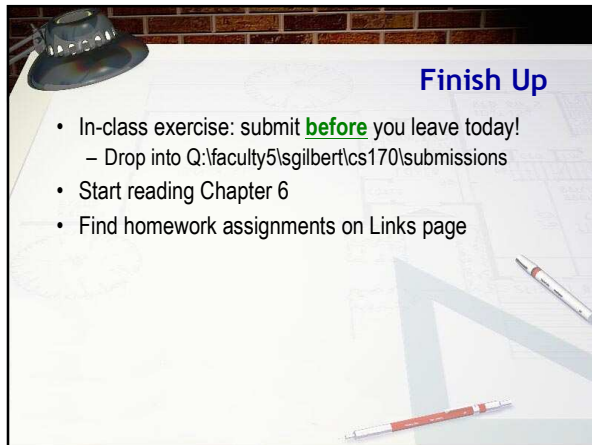
- Exercise 12:** change the world to `castle2`, patrol the castle and shoot a picture.



Express Yourself

- Create a Robot to walk the fixed maze shown here:
 - MazeWalker extends OCRobot, add any methods
 - Make sure you enter at 1,3 and leave at 9,9
 - Walk entire maze, pick up beepers where you find them
 - Change URL to "lab02h.php" and run again to make sure you get all beepers.





Finish Up

- In-class exercise: submit **before** you leave today!
 - Drop into Q:\faculty5\sgilbert\cs170\submissions
- Start reading Chapter 6
- Find homework assignments on Links page
